

Xây dựng Framework Tạo Mô hình AI cho Ứng dụng Theo dõi Chuyển động Con người

Luận văn tốt nghiệp — Bảo vệ luận văn

Sinh viên: Nguyễn Trương Minh Hoàng (MSSV 2270757)

GVHD: TS. Lê Trọng Nhân

Khoa Khoa học & Kỹ thuật Máy tính
Trường Đại học Bách Khoa TP.HCM

Tháng 6 năm 2026

Nội dung trình bày

- ❶ Bối cảnh, bài toán và mục tiêu nghiên cứu
- ❷ Framework đề xuất và quy trình hoạt động
- ❸ Thiết lập thực nghiệm và kết quả chính
- ❹ Kết quả triển khai thiết bị và so sánh nền tảng
- ❺ Kết luận, giới hạn và hướng phát triển

Mục tiêu trình bày 20 phút: tập trung vào bài toán, giải pháp, bằng chứng thực nghiệm và giá trị thực tiễn của framework.

Bối cảnh & Bài toán

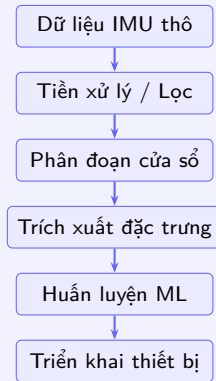
HAR dựa trên IMU có nhu cầu cao trong:

- **Y tế:** phát hiện té ngã, phục hồi chức năng
- **Thể thao:** phân tích kỹ thuật, đếm bài tập
- **Công nghiệp:** phát hiện tư thế nguy hiểm

Yêu cầu triển khai thực tế:

- suy luận trực tiếp trên thiết bị biên
- độ trễ thấp, tiết kiệm năng lượng
- không phụ thuộc kết nối mạng

Khoảng trống nghiên cứu



Phần lớn công trình dừng ở bước huấn

Mục tiêu nghiên cứu & Câu hỏi chính

Câu hỏi trung tâm

Có thể xây dựng một framework mã nguồn mở giúp người dùng đi từ dữ liệu IMU thô đến mã suy luận chạy trên vi điều khiển một cách nhất quán, minh bạch và tái sử dụng được hay không?

Mục tiêu:

- Tạo pipeline end-to-end cho HAR trên IMU
- Giảm rào cản kỹ thuật ở bước tiền xử lý và triển khai
- Đảm bảo tính tương đồng giữa huấn luyện và suy luận thiết bị
- Hỗ trợ nhiều mô hình và nhiều đích triển

Phạm vi đánh giá:

- Dữ liệu IMU 6 trục, 100 Hz
- 5 hoạt động người dùng
- RF, SVM, NN, PyTorch MLP, CNN
- Thiết bị tham chiếu: Seede XIAO nRF52840 Sense

Đóng góp chính của luận văn

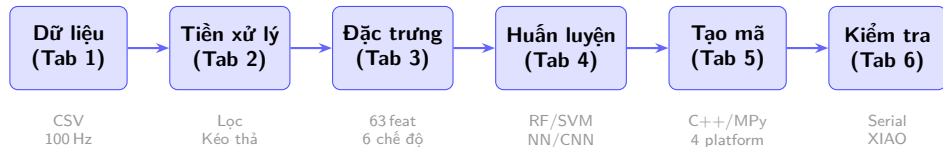
Về hệ thống:

- ✓ Framework end-to-end: dữ liệu → đặc trưng → huấn luyện → mã triển khai
- ✓ Giao diện kéo thả cho tiền xử lý và phân đoạn cửa sổ
- ✓ Lưu trữ artifact huấn luyện theo dạng dùng lại được

Về triển khai:

- ✓ Bảo đảm parity Python ↔ C++ trong pipeline trích xuất đặc trưng
- ✓ Hỗ trợ RF, SVM, NN, PyTorch MLP và CNN
- ✓ Sinh mã cho Arduino-compatible, ARM Cortex-M, Zephyr, MicroPython
- ✓ Xác thực bằng kết quả huấn luyện và thử nghiệm trên thiết bị thực

Kiến trúc Framework



Công nghệ

Python 3 · Dash 2.14
scikit-learn · PyTorch
joblib · pandas · numpy

Lưu trữ trạng thái

persistent_data/
CSV → joblib → .h/.ino
Không cần database

Thiết bị tham chiếu

Seeed XIAO nRF52840
IMU LSM6DS3, 6 trục
ARM Cortex-M4F 64 MHz

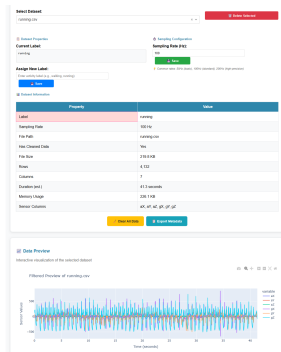
Thiết lập thực nghiệm

Dữ liệu và phần cứng:

- IMU 6 trục, 100 Hz, thiết bị Seeed XIAO nRF52840 Sense
- 5 hoạt động: running, still, walking, walking downstairs, walking upstairs
- 2 cấu hình đánh giá: bài toán 3 lớp và 5 lớp

Cấu hình xử lý:

- Cửa sổ 150 mẫu (1,5 s), bước trượt 75 mẫu
- Đặc trưng mặc định: `orientation_invariant` (63 đặc trưng)
- CNN dùng trực tiếp đầu vào thô 150×6
- Chia dữ liệu: train / validation / test



Giao diện quản lý dữ liệu đầu vào của framework

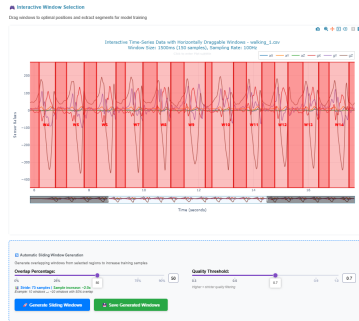
Tiền xử lý & Trích xuất đặc trưng

Tiền xử lý tương tác:

- Kéo thả trực tiếp vùng hoạt động trên biểu đồ tín hiệu
- Hỗ trợ các bộ lọc Butterworth, Savitzky-Golay, Kalman
- Đệm cửa sổ bằng lặp giá trị biên để giữ tính hợp lý vật lý

Trích xuất đặc trưng:

- 6 chế độ đặc trưng phục vụ nhiều mức tài nguyên khác nhau
- Chế độ mặc định: `orientation_invariant`
- 15 thống kê/tín hiệu: mean, std, min,



Giao diện tiền xử lý và phân đoạn cửa sổ kéo thả

Thông điệp chính: framework cho phép người dùng kiểm soát trực tiếp bước tiền xử lý thay vì xem đó là một hộp đen.

Huấn luyện mô hình & Tạo mã triển khai

Các mô hình được hỗ trợ:

- Random Forest
- SVM (RBF)
- Neural Network (scikit-learn)
- PyTorch MLP
- PyTorch CNN

Ý tưởng triển khai:

- Model sau huấn luyện được lưu thành artifact thống nhất
- Từ artifact này, framework sinh mã cho nhiều nền tảng khác nhau
- Người dùng không cần viết tay phần

Nền tảng đích được hỗ trợ:

- Arduino-compatible (XIAO, ESP32, M5Stack, Teensy)
- ARM Cortex-M generic (C không phụ thuộc thư viện)
- ESP-IDF, Zephyr RTOS
- MicroPython (không cần NumPy)
- TFLite Micro / ONNX Runtime (NN/CNN)

4 chế độ tối ưu:

- accuracy: 6 chữ số thập phân, debug bật
- balanced: 3 chữ số (mặc định)
- speed: 2–3 chữ số, tối ưu bộ đệm

Kết quả Thực nghiệm

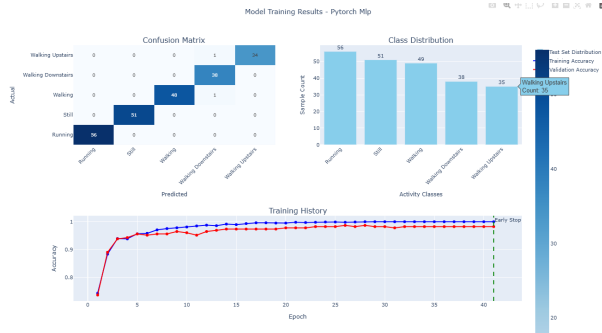
Thiết lập chính: IMU 6 trục, 100 Hz, cửa sổ 150 mẫu, bước 75, đặc trưng orientation_invariant.

Bài toán 5 lớp (229 mẫu kiểm tra)

Thuật toán	Acc. (%)	F1
PyTorch MLP	99,13	0,990
PyTorch 1D-CNN	99,13	0,992
Neural Network	98,69	0,984
Random Forest	98,25	0,981
SVM (RBF)	97,38	0,972

Nhận xét: chênh lệch giữa mô hình tốt nhất và thấp nhất chỉ 1,75 điểm %, cho thấy pipeline đặc trưng có chất lượng tốt.

Ghi chú ngắn: bài toán 3 lớp đạt 100% trên cả 5 thuật toán, chứng tỏ framework hoạt động



Mã trận nhầm lẫn của PyTorch MLP: lỗi chủ yếu xuất hiện giữa các lớp đi bộ và cầu thang.

Kết quả Triển khai Thiết bị (Seeed XIAO nRF52840)

Kết quả triển khai:

- PyTorch 1D-CNN: **ổn định cả 3 và 5 lớp**
- NN sklearn: lỗi phân loại running trong một số điều kiện
- Random Forest: ổn định cho 3 lớp
- Ngưỡng confidence: loại bỏ dự đoán không chắc chắn

Yêu cầu tài nguyên (Neural Network 5 lớp):

- Flash: ~95 KB (bao gồm trọng số + FE + scaler)
- SRAM: <8 KB (bộ đệm cửa sổ + tính toán)

So sánh với TFLite Micro

	Framework	TFLite
Flash (NN)	95 KB	~245 KB
RF hỗ trợ	✓	✗
SVM hỗ trợ	✓	✗
Mã có thể đọc	✓	✗

Thông điệp chính

Độ chính xác ngoại tuyến là cần thiết nhưng chưa đủ; giá trị thực của framework nằm ở khả năng đưa mô hình ra thiết bị với quy trình có thể kiểm tra và tái sử dụng.

So sánh với Nền tảng Thương mại

Tiêu chí	Edge Impulse	SensiML	Framework này
Chi phí	\$20–99/tháng	\$99–500/tháng	Miễn phí
Mã nguồn mở	✗	✗	✓
Minh bạch pipeline triển khai	Hộp đen	Hộp đen	✓ Toàn bộ
Hỗ trợ RF/SVM	Giới hạn	Có chọn lọc	✓
Đảm bảo parity	Không rõ	Không rõ	✓ Bit-exact
F1 trên 3 lớp	0,96	—	1,00
Flash (NN)	51%	—	18–23%

Điểm mạnh độc đáo

Giá trị khác biệt của framework là **minh bạch + tái sử dụng + hỗ trợ nhiều mô hình**, thay vì chỉ tối ưu trải nghiệm kéo-thả như các nền tảng thương mại.

Kết luận & Hướng phát triển

Đã đạt được:

- ✓ Framework end-to-end, mã nguồn mở
- ✓ 97,38–99,13% (5 lớp), 100% (3 lớp)
- ✓ Tương đồng huấn luyện–triển khai được bảo đảm
- ✓ Triển khai thành công trên XIAO nRF52840
- ✓ Vượt Edge Impulse: F1 1,00 vs 0,96

Giới hạn và hướng phát triển:

- Hiện mới đánh giá sâu trên dữ liệu một đối tượng và thiết bị tham chiếu
- Cần mở rộng sang dữ liệu đa người dùng, đa hướng gắn cảm biến
- Có thể bổ sung học liên tục, sensor fusion và tối ưu đa mục tiêu

Kết luận tổng quát

Luận văn chứng minh rằng có thể xây dựng một framework HAR mã nguồn mở giúp rút ngắn khoảng cách từ huấn luyện đến triển khai thiết bị biên, đồng thời vẫn giữ được tính minh bạch và khả năng kiểm chứng.

Cảm ơn Quý Thầy/Cô & Hội đồng

Xin mời Hội đồng đặt câu hỏi

Sinh viên: Nguyễn Trương Minh Hoàng

MSSV: 2270757

GVHD: TS. Lê Trọng Nhân

Đề tài: Framework AI cho HAR trên thiết bị biên

Xin cảm ơn Quý Thầy/Cô và Hội đồng đã lắng nghe.

[Dự phòng] Nhất quán giữa huấn luyện và triển khai

Các nguyên tắc đảm bảo nhất quán:

- 1 Cùng công thức đặc trưng ở Python và C++
- 2 Cùng thứ tự đặc trưng khi huấn luyện và sinh mã
- 3 Cùng tốc độ lấy mẫu, cửa sổ và chuẩn hóa
- 4 Artifact mô hình chứa metadata để tái sử dụng khi triển khai
- 5 Các mô hình raw-input như CNN có đường triển khai riêng

Ý nghĩa thực tiễn:

Nếu không kiểm soát tính nhất quán, mô hình có thể đạt kết quả tốt trên máy tính nhưng hoạt động không ổn định khi đưa lên MCU.

Framework giải quyết điều này bằng cách giữ toàn bộ chuỗi xử lý trong cùng một pipeline có thể đọc, kiểm tra và tái sử dụng.

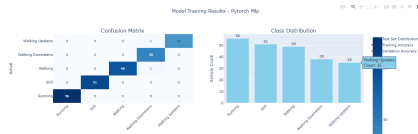
[Dự phòng] Phân tích Ma trận Nhầm lẫn

Bài toán 5 lớp — PyTorch MLP (F1 = 0,990):

Hoạt động	Precision	Recall
Running	1,000	1,000
Still	1,000	1,000
Walking	1,000	0,980
W. Downstairs	0,950	1,000
W. Upstairs	1,000	0,971
Macro	0,990	0,990

Nhận xét:

- Running, Still: F1 = 1,000 (đặc trưng biên độ phân biệt rõ)
- Lỗi tập trung ở nhóm đi bộ (walking vs stair variants)
- 1 mẫu walking bị nhầm → W.Downstairs (precision 0,950)
- Orientation-invariant feature giúp phân biệt tốt hơn per-axis feature trước đây



[Dự phòng] Câu hỏi thường gặp

Q: Tại sao không dùng TFLite Micro cho tất cả?

A: RF/SVM không có đường chuyển đổi sang TFLite (ONNX-ML operators không được onnx2tf hỗ trợ). Framework hỗ trợ cả hai hướng: tạo mã trực tiếp (tất cả mô hình) + TFLite path (chỉ NN/CNN).

Q: Dataset chỉ 1 đối tượng có đủ để đánh giá?

A: Mục tiêu luận văn là *framework*, không phải xây dựng mô hình HAR tổng quát cuối cùng. Dữ liệu hiện tại đủ để chứng minh pipeline hoạt động end-to-end; mở rộng đa đối tượng là bước tiếp theo.

Q: Làm sao biết parity được đảm bảo?

A: Framework có validation script (`validate_deployment.py`) để so sánh đặc trưng Python và C++ trên cùng dữ liệu đầu vào, đồng thời lưu metadata mô hình để tái sử dụng nhất quán khi sinh mã.